SecureLand: Property Document Management, Integrity Verification, and Administrative Review

A Project Report Submitted for Academic Evaluation

Abstract

SecureLand is a comprehensive web application engineered for the secure submission, management, and review of property-related documents. The system allows authenticated users to upload supported files and monitor their document records alongside review statuses. Furthermore, administrators are equipped to manage trusted reference documents, compare uploaded files with active references utilizing server-side SHA-256 cryptographic hashes, and systematically approve or reject submissions. All verification attempts and administrative decisions are meticulously recorded in an audit log for subsequent review.

Crucially, the system architecture purposefully decouples document administrative approval from technical file-integrity results. A hash match strictly indicates byte-for-byte equivalence; it does not inherently prove property ownership, establish legal validity, or authenticate an issuing authority. To maintain stringent security, documents are strictly stored outside the public web directory and are solely retrieved through authenticated, role-based server endpoints.

Table of Contents

1. Introduction 4

2. Problem Statement 4

3. Existing System and Limitations 4

4. Proposed System 5

5. Project Objectives 5

6. Scope and Features 6

6.1 User Features 6

6.2 Administrator Features 6

7. System Architecture 7

8. Main Data Entities 7

9. Typical Workflows 8

10. Security and Data Integrity 9

11. Technology Stack 9

12. Expected Results and Advantages 10

13. Limitations and Future Enhancements 10

14. Conclusion 10

1. Introduction

Property documents are critical records that require stringent oversight, secure storage, and transparent workflows. As digital transformation accelerates, a robust digital document workflow is required to protect access while making review statuses unequivocally clear to both users and administrators. SecureLand provides a centralized, authenticated interface for document submission, controlled administrative review, and technical comparison against registered trusted reference files. The application is implemented as a modern React and TypeScript frontend paired with a highly secure PHP backend and a MySQL-compatible relational database.

2. Problem Statement

Managing uploaded property documents without a consistent, technologically enforced workflow makes it difficult to track file provenance, submission status, and data integrity. Traditional methods struggle to answer critical questions: who submitted a file, is it awaiting review, and do its exact bytes match a known trusted reference? SecureLand directly addresses these workflow and integrity-checking needs while ensuring a strict logical separation between subjective administrator approval and objective technical hash comparisons.

3. Existing System and Limitations

In existing document management paradigms, organizations frequently rely on disjointed silos—such as basic email attachments, localized physical storage, or rudimentary shared drives. These systems present significant limitations:

- **Lack of Integrity Verification:** Traditional systems rarely employ cryptographic hashing to verify if a submitted document has been altered from its original form.
- **Ambiguous Workflows:** Submissions often fall into a "black hole," leaving users unaware of their document's review or approval status.
- **Security Vulnerabilities:** Documents are frequently stored in easily accessible public directories or without strict Role-Based Access Control (RBAC).
- **Insufficient Audit Trails:** Manual systems fail to reliably record who reviewed a document, when it was verified, and the specific reason for its approval or rejection.

4. Proposed System

The proposed system, SecureLand, serves as a comprehensive digital solution that centralizes and secures the property document lifecycle. By leveraging modern web technologies (React/TypeScript) and a robust backend (PHP/MySQL), the system enforces strict authentication and authorization protocols. It introduces a mechanism where uploaded documents are technically compared against trusted reference files using server-side SHA-256 hash calculations. This allows administrators to verify data integrity with mathematical certainty before applying human administrative judgment to approve or reject the submission.

5. Project Objectives

The primary objectives of the SecureLand project are defined as follows:

- Provide highly secure, authenticated access for both general users and administrators.
- Accept supported property-document uploads safely by storing them strictly outside the public web directory.
- Maintain detailed document metadata, user ownership structures, upload timelines, and review statuses within a relational database.
- Calculate SHA-256 cryptographic fingerprints on the server to accurately compare user documents with active trusted references.
- Maintain a distinct separation between objective integrity-check results and subjective administrator approval decisions.
- Empower administrators to review, approve, or reject pending submissions, enforcing mandatory reasoning for rejections.
- Preserve a comprehensive verification history and immutable administrative audit records.
- Implement foundational security measures including session authentication, role checks, CSRF protection, and prepared SQL statements.

6. Scope and Features

The scope of SecureLand encompasses comprehensive feature sets tailored for two primary user roles: general users submitting documents, and administrators managing the verification and approval lifecycle.

6.1 User Features

- **Authentication:** Securely log in and maintain an authenticated session.
- **Document Upload:** Upload PDF, JPEG, or PNG property documents, with the interface actively enforcing a strict 10 MB maximum file size limit.
- **Metadata Management:** Provide extensive document and property metadata, including document number, property ID, legal owner, issuing authority, issue date, and contextual descriptions.
- **Dashboard Tracking:** View owned documents, deep details, upload dates, current approval statuses, and available technical verification information.
- **History Access:** Access verification history strictly subject to ownership validation or explicit administrator authorization.
- **Feedback Loop:** View explicit rejection reasons for denied documents directly through the user-facing workflow interface.

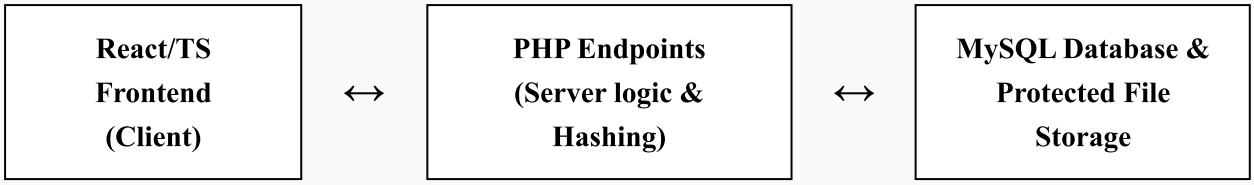
6.2 Administrator Features

- **Analytical Dashboard:** View dynamic, database-backed dashboard statistics regarding system throughput and document statuses.
- **Queue Management:** Systematically review documents residing in the pending-verification queue.
- **Secure Retrieval:** Open or download submitted documents exclusively through authorized, access-controlled backend endpoints.
- **Integrity Comparison:** Compare an uploaded document with an active trusted reference utilizing server-side hash generation.
- **Reference Management:** Register and manage trusted baseline references, including toggling their activation state via protected endpoints.
- **Decision Enforcement:** Approve or reject pending documents. A mandatory reason is required for rejection, and the reviewer’s identity alongside the exact timestamp are permanently recorded.
- **Audit Oversight:** Review comprehensive administrative audit logs and historical verification records.

7. System Architecture

SecureLand follows a distinct client/server architecture model. The single-page React frontend communicates seamlessly with the PHP backend via JSON-based RESTful API endpoints. In the development environment, Vite acts as the frontend server and proxies API requests securely to the PHP server environment.

Figure 1: High-Level System Architecture



8. Main Data Entities

The core data is structured within a MySQL-compatible relational database, utilizing the following primary entities to ensure data normalization and integrity:

Entity Name	Purpose and Description
`users`	Stores user identities, highly secure password hashes, and system roles (`user` or `admin`).
`secureland_documents`	Stores document ownership mappings, generated filenames, calculated SHA-256 hashes, user-provided metadata, upload dates, processing states, and review audit metadata.
`secureland_trusted_references`	Maintains registered trusted reference metadata, definitive SHA-256 hashes, activation/validation states, and administrator tracking fields.
`secureland_verification_logs`	Records immutable verification results, the selected reference file, the verifying administrator's identity, precise timestamps, and hash snapshots/reason details.
`secureland_admin_audit_log`	Provides a tamper-evident record of administrative actions, specifically tracking document-review decisions.

9. Typical Workflows

The application defines strict procedural workflows to ensure that all document manipulations are intentional, secure, and properly logged.

9.1 User Upload Workflow

1. **Authentication:** The user successfully authenticates and establishes a secure PHP session.
2. **Data Entry:** The user selects a supported document format (PDF, JPEG, PNG) and inputs all required property metadata.
3. **Validation and Storage:** The backend server rigorously validates the incoming request, inspects the file payload, and securely writes it to a directory located safely outside the public web root.
4. **Cryptographic Hashing:** The server dynamically calculates the file's SHA-256 mathematical hash and initializes a new document record tied to the authenticated user's ID.
5. **Queue Entry:** The document is immediately placed into the pending approval workflow. Crucially, a successful upload and successful hash calculation do not inherently approve the document for use.

9.2 Administrator Review Workflow

1. **Queue Access:** An administrator logs in and accesses the global pending-document verification queue.
2. **Inspection:** The administrator inspects the submitted metadata and utilizes authorized endpoints to securely retrieve and view the file.
3. **Reference Comparison:** The administrator triggers a technical comparison of the uploaded file against a known, active trusted reference.
4. **Integrity Result:** The server recalculates and compares both hashes, formally recording the verification attempt. A missing reference or an unmatched hash string explicitly registers as a failed integrity check.
5. **Decision Making:** Using human judgment alongside the technical hash result, the administrator approves or rejects the document. The system validates the administrator's session, role, CSRF token, and the document's current pending status before persisting the decision.
6. **User Notification:** The user's dynamic dashboard updates to reflect the new approval status and associated administrative reasoning.

10. Security and Data Integrity

Security is the foundational principle of the SecureLand application architecture. The system implements defense-in-depth methodologies across all layers:

- **Session Management:** Native PHP sessions authenticate users. To prevent session fixation, the login process explicitly regenerates the session identifier.
- **Access Control:** Strict Role-Based Access Control (RBAC) checks protect all administrator endpoints. Document details and downloads are strictly restricted to the proven owner or an authorized administrator.
- **CSRF Mitigation:** Cryptographic Cross-Site Request Forgery (CSRF) tokens are validated on all state-changing operations.
- **Storage Security:** Uploaded content is verified by MIME type and size. Stored filenames are cryptographically generated by the server rather than trusting user input. Files are physically stored outside the public web root, completely preventing direct URL access.
- **Database Security:** PHP Data Objects (PDO) prepared SQL statements are utilized exclusively to eliminate SQL injection vulnerabilities.
- **Credential Security:** Passwords are cryptographically hashed before storage; plain-text passwords are never compared or stored.
- **Data Integrity:** The SHA-256 algorithm calculates hashes server-side from raw file bytes. Importantly, this is utilized for mathematical integrity comparison, not encryption. System architecture ensures that administrative approval, technical integrity, and digital-signature states remain entirely independent concepts.

11. Technology Stack

SecureLand is built upon a modern, highly reliable technology stack:

- **Frontend:** React 19, TypeScript, React Router for navigation, Tailwind CSS for responsive styling, and Vite for optimal build tooling.
- **Backend:** PHP 8+ utilizing native PHP session management, file-upload handling, native hashing, and PDO APIs. The backend eschews third-party Composer packages in favor of native PHP capabilities to minimize the supply chain attack surface.
- **Database:** MySQL/MariaDB accessed securely through the PDO MySQL driver.
- **Development & QA:** The TypeScript compiler and Oxlint are used to enforce code quality and type safety prior to deployment.

12. Expected Results and Advantages

The successful implementation of SecureLand yields a highly functional, secure environment for managing property documents. The primary advantages over traditional systems include immediate tamper detection via SHA-256 hashing, comprehensive and immutable audit trails mapping exactly who performed which action, and absolute transparency regarding a document's position within the administrative review pipeline. The system enforces organizational policy through code, dramatically reducing the risk of forged document acceptance.

13. Limitations and Future Enhancements

While the current build is highly secure, specific features are classified as out of scope for this iteration and represent opportunities for future development:

- **Encryption at Rest:** While protected storage and strict access controls are implemented, advanced AES file-at-rest encryption is not currently active.
- **Digital Signatures:** The parsing of PDF digital signatures and underlying cryptographic signature validation are not implemented.
- **Blockchain Integration:** The application does not currently implement a distributed blockchain or an immutable cryptographic hash chain to decentralize trust.
- **Hash Scope:** A matching SHA-256 hash strictly establishes byte equality with a reference; it cannot programmatically establish legal ownership or the authenticity of the issuing authority.

14. Conclusion

SecureLand effectively demonstrates a highly controlled property-document workflow prioritizing authenticated uploads, ownership-aware access, strict administrator review, trusted-reference management, and server-side SHA-256 mathematical comparison. By intelligently separating technical file byte-equality from subjective administrative approval and legal claims, the system provides a realistic, robust, and highly auditable framework for modern digital document management.

References

- React Documentation: https://react.dev/
- TypeScript Documentation: https://www.typescriptlang.org/docs/
- PHP Official Documentation: https://www.php.net/docs.php
- MySQL Database Documentation: https://dev.mysql.com/doc/
- Vite Tooling Guide: https://vite.dev/guide/